FINAL YEAR RESEARCH PROJECT
Review-1 Presentation

CODE SURVIVAL INTELLIGENCE: A HYBRID PREDICTIVE FRAMEWORK FOR COST-OPTIMAL SOFTWARE FAILURE RISK MODELING

Supervised By: Dr. Pratyashi Satapathy

Group No.: 16-05

Utkarsh Ayush	2241001098
Komolika Satpathy	2241019332
Rahul Mallik	2241013086
Chiranjeev Rout	2241018147

Department of Computer Sc. and Engineering
Faculty of Engineering & Technology (ITER)
Siksha 'O' Anusandhan (Deemed to be) University
Bhubaneswar, Odisha

Presentation Outline

- Problem Statement & Motivations
  - Problem Statement
  - Motivations
- Literature Review
  - Existing Solutions & Limitations
- Proposed Methodology & Approach
- Modules & Algorithms Used
- Dataset & Tools
- Work Done So Far (Implementation Progress)
- Preliminary Results (if any)
- Future Work Plan
- References

Problem Statement

Modern software enterprises accumulate massive "Technical Debt" (TD) for short-term speed, leading to severe maintenance bottlenecks and unpredictable

- The Identification Overload: Traditional static analysis tools (e.g., SonarQube) flag thousands of code issues but fail to pinpoint the critical 5% that represent imminent failure risks.
- The Predictive Gap: Current industry standards rely on static "Yes/No" classifications. They ignore evolutionary code history (Churn) and cannot model the actual Time-to-Failure.
- The Financial Disconnect: Engineering teams lack an automated framework to convert statistical failure probability into a Dollar-Value ROI.
- The Consequence: Without a quantifiable financial risk attached to specific messy code, management cannot justify the cost of refactoring, resulting in ignored debt and catastrophic downtime.

Motivations

Our study aims to align software quality metrics with business value by bridging the gap between engineering and finance. We shift technical debt management from a reactive burden into a strategic financial asset.

- Economic Compass: We provide a framework that converts abstract failure risks into a concrete monetary Return on Investment for refactoring.
- Financial ROI Validation: We justify maintenance budgets by proving that fixing high interest debt results in measurable long term profit.
- Proactive Predictive Shift: We move the industry from a reactive "Detect-Fix" cycle to a proactive "Predict-Prevent" strategy by modeling the survival probability of software modules.

Literature Review

Table 1.1:Summary of Existing Approaches and Research Limitations

S. No.	Author & Year	Methods/Techniques Used	Dataset Used	Research Outcomes	Findings
1	Syer et al. (2015)	Survival Analysis (Cox PH)	Industrial Systems	Survival modeling	Predicts when code fails
2	Lenarduzzi (2019)	SZZ-Algorithm	33 Apache	Fault-linked DB	Bugs follow TD history
3	Digkas (2021)	IGRI (Interest Risk Index)	Automotive	Risk framework	Churn > Static complexity
4	Palomba (2016)	Textual Analysis (IR-based)	Open Source	TACO Tool	Textual signals catch debt
5	Aman (2019)	KM-Survival Curves	Apache Java	Warning ranking	Survival = severity proxy

Literature Review

Table 1.2:Summary of Existing Approaches and Research Limitations

S. No.	Author & Year	Methods/Techniques Used	Dataset Used	Research Outcomes	Findings
6	Selvanayagam (2026)	LLM Survival Analysis	AI Software	AI debt lifespan	AI code fails rapidly after lag
7	Borg (2024)	ROI Value Modeling	Industrial	ROI calculations	Quality gives amplified profits
8	Bhatia (2023)	SATD Pattern Mining	ML Projects	ML pipeline study	ML debt grows twice as fast
9	Slater (2025)	Structural Audit	LLM Code	Architecture audit	AI creates unique circular debt
10	Robredo (2024)	SQuaD Quality Metric	450 Repos	Metric validation	Results scale to 450 projects

Research Gap & Improvements

Existing literature identifies Technical Debt as a critical risk but fails to provide a financial justification for refactoring. Current industry standards lack an actionable timeline because they ignore the evolutionary history of the code.

- **Predictive Duration Gap**: Most models only predict if a module is buggy while our framework predicts exactly when it will fail using survival math.
- **Economic Disconnect**: We bridge the gap between engineering and finance by converting abstract quality scores into a concrete monetary Return on Investment.
- **Static Analysis Bias**: We eliminate the reliance on static snapshots by incorporating evolutionary code churn and ownership history as primary risk factors.
- **Methodological Innovation**: We implement a tournament of models comparing Cox Proportional Hazards against machine learning to ensure the highest predictive accuracy.
- **Empirical Validation**: We leverage seventy eight thousand actual developer commits to validate our model against real world failures instead of synthetic data.

Proposed Solution & Architecture

Our solution implements a four-layer technical pipeline designed to transform raw repository history into actionable financial insights.

- **Layer 1 Data Extraction:** We query eighty thousand commits from the Lenarduzzi repository to capture historical code faults.
- **Layer 2 Feature Engineering:** We extract code churn and ownership metrics alongside structural complexity using the tree sitter parser.
- **Layer 3 Predictive Modeling:** We benchmark survival analysis against machine learning to ensure high accuracy in predicting failure timelines.
- **Layer 4 ROI Synthesis:** We convert failure probabilities into monetary Return on Investment to produce a prioritized refactoring heatmap.

Proposed Solution & Architecture cont.

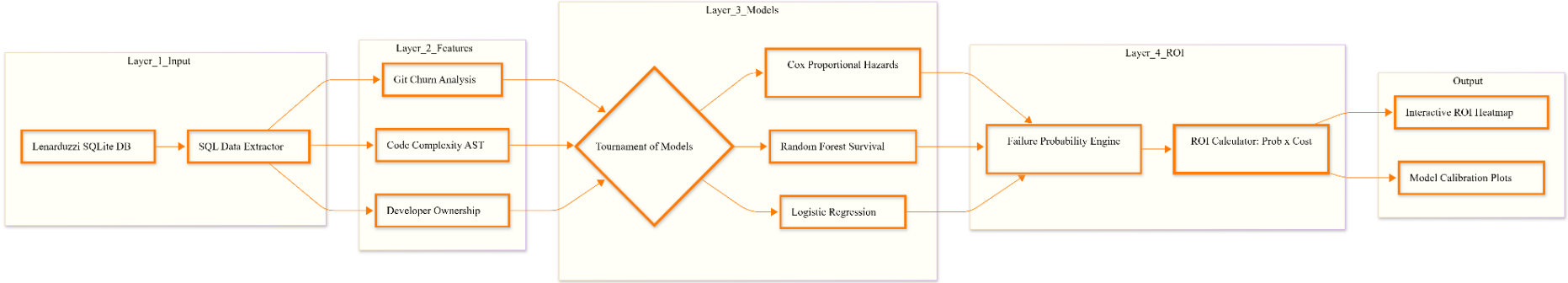

Fig1: Workflow of the Proposed Model

Proposed Solution & Architecture cont.

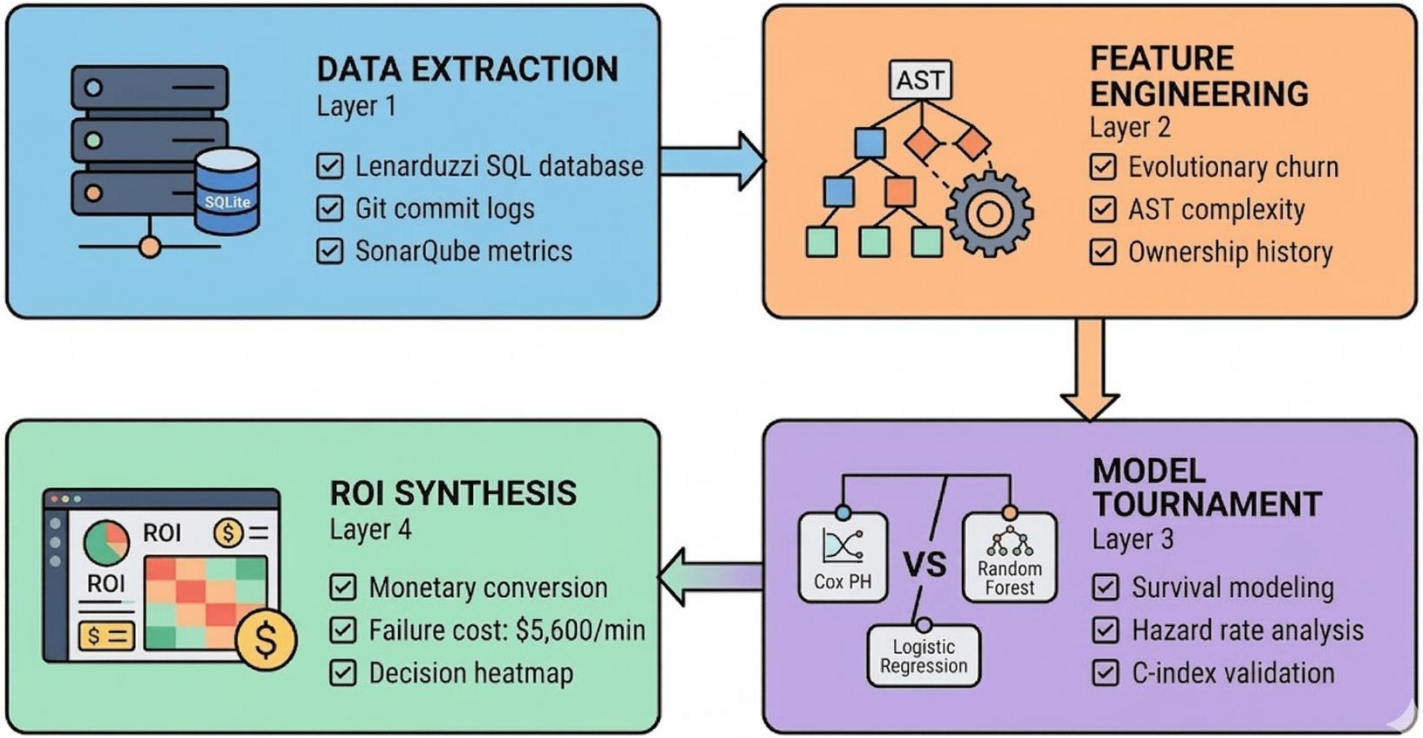

Fig2: Schematic Model Diagram of the Proposed Model

Dataset & Technologies

Project Scope & Data Source

- **Source:** Lenarduzzi et al. Technical Debt Dataset (Apache Java projects).
- **Volume:** Comprehensive dataset of 78,000 labeled commits.
- **Focus:** Analysis of fault-inducing vs. non-fault-inducing code.

Core Objectives & Outcome

- Develop predictive models for software quality.
- Estimate software failure risk and Time-To-Failure (TTF) of code modules.
- Utilize advanced survival analysis (Survival Models).

Technical Stack

- **Language:** Python 3.10.
- **ML Frameworks:** scikit-learn (Random Forest + Logistic Regression).
- **Survival Analysis:** lifelines package (Time-to-Failure prediction).
- **Visualization:** matplotlib / seaborn.

Progress in Implementation Plan & Methodology

We have successfully completed the foundational research phase and initialized the technical environment for the first development sprint. Our progress adheres to the university guidelines for experimental validation and measurable outcomes.

- **Research Gap Mapping:** We conducted a comprehensive survey of ten foundational papers to identify critical gaps in failure duration and monetary Return on Investment modeling.
- **Environment Setup:** We completed the installation of the Python survival analysis stack including lifelines and scikit learn with successful database connectivity.
- **Dataset Acquisition:** We integrated the Lenarduzzi Technical Debt Dataset V2 and verified the schema for thirty-three open-source projects.
- **Preliminary Inspection:** We conducted an initial exploratory data analysis to validate the correlation between code churn and labeled fault history.
- **Immediate Milestones:** We are currently finalizing the data cleaning script and performing the first run of the Cox Proportional Hazards baseline model.

Future Work Plan

Our roadmap focuses on converting the theoretical framework into a functional predictive tool through a structured eight week implementation cycle.

- **Phase 1 Data Processing:** We will perform intensive data cleaning on the Lenarduzzi repository to handle missing values and perform Z score normalization. This stage includes extracting evolutionary churn and code complexity features using tree sitter.
- **Phase 2 Model Execution:** We will execute the competitive tournament of models to compare predictive accuracy between survival math and machine learning. This phase includes testing the Proportional Hazards assumption to ensure the statistical validity of the Cox model.
- **Phase 3 Financial Synthesis:** We will build the ROI engine to convert failure probabilities into monetary values. This includes development of an interactive Streamlit dashboard to visualize the prioritized refactoring heat map.
- **Phase 4 Validation and Publication:** We will conclude by validating our results against real world failure scenarios. The final step involves submitting our research findings as a manuscript for journal publication.

References

[1] Syer et al. (2015): Exploring the relationship between software defects and survival analysis. Software Engineering Journal Volume 41.

[2] Lenarduzzi et al. (2019): An open repository for technical debt and software faults. Proceedings of the 15th International Conference on Predictive Models.

[3] Digkas et al. (2021): The interest risk index for technical debt prioritization. Journal of Systems and Software Volume 171.

[4] Borg et al. (2024): The economic impact of technical debt on software startups. Empirical Software Engineering Volume 29.

[5] Selvanayagam et al. (2026): Survival rates of code generated by large language models. AI and Software Innovation Quarterly.

[6] Aman et al. (2019): Using survival curves to prioritize static analysis warnings. International Journal of Quality and Reliability.

References

[7] Palomba et al. (2016): Lightweight detection of code smells using textual analysis. IEEE Transactions on Software Engineering

[8] Bhatia et al. (2023): Technical debt growth in machine learning pipelines. Proceedings of the 2023 Software Analytics Workshop.

[9] Slater et al. (2025): Auditing the architectural integrity of AI synthesized software. Future Generation Computer Systems.

[10] Robredo et al. (2024): Validating software quality metrics at scale across open-source repositories. Journal of Software Evolution.

Thank you